

ES -116

Principles and Applications of Electrical Engineering

Semester II, 2025-26

How Processors Compute, Direct, and Store Data?

Reference book: Digital Systems Book by Tocci

Recap: All we wanted to do to add $1 + 1$ in binary!

$$1 + 1 = 2$$

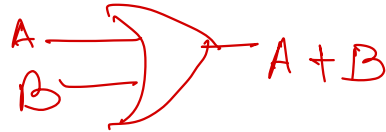
Add $1+1$ in binary representation (What hardware do I need it to do that?)

$$A \rightarrow 1 + 1 \leftarrow B = 0$$

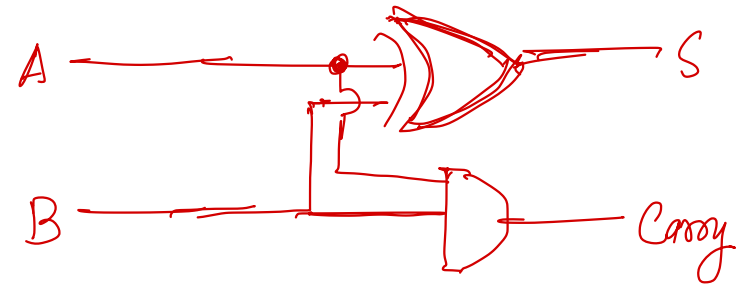
operands

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

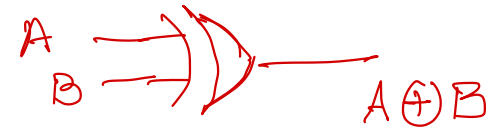
$$\begin{array}{r} A_1 + B_1 + C_0 \\ \downarrow \\ C_0 \\ \leftarrow A_1, A_0 \\ 0 \ 1 \\ \leftarrow B_1, B_0 \\ 0 \ 1 \\ \hline 1 \ 0 \leftarrow S_1, S_0 \end{array}$$



1-bit addition



Half Adder

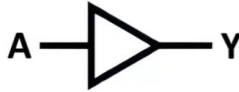


Concepts Review (1)

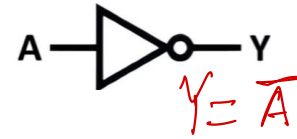
Types of Logic Gates

NOT, AND, OR

BUFFER Gate	
A	Y
0	0
1	1

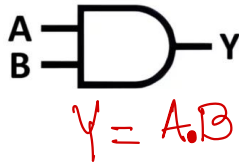


NOT Gate	
A	Y
0	1
1	0



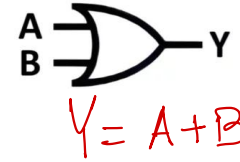
$$Y = \overline{A}$$

AND Gate		
A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1



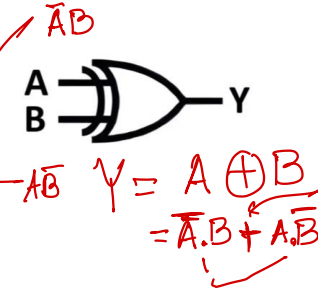
$$Y = A \cdot B$$

OR Gate		
A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1



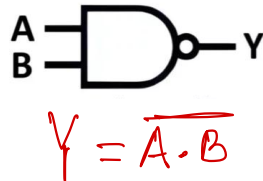
$$Y = A + B$$

XOR Gate		
A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0



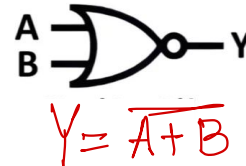
$$Y = A \oplus B$$
$$= \overline{A} \cdot B + A \cdot \overline{B}$$

NAND Gate		
A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0



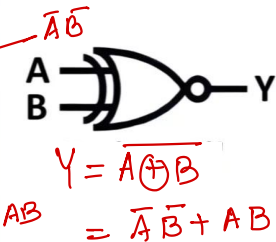
$$Y = \overline{A \cdot B}$$

NOR Gate		
A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0



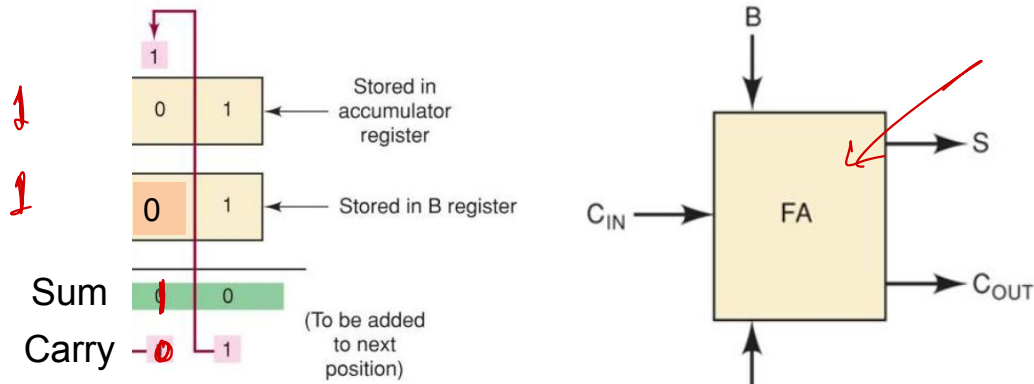
$$Y = \overline{A + B}$$

XNOR Gate		
A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1



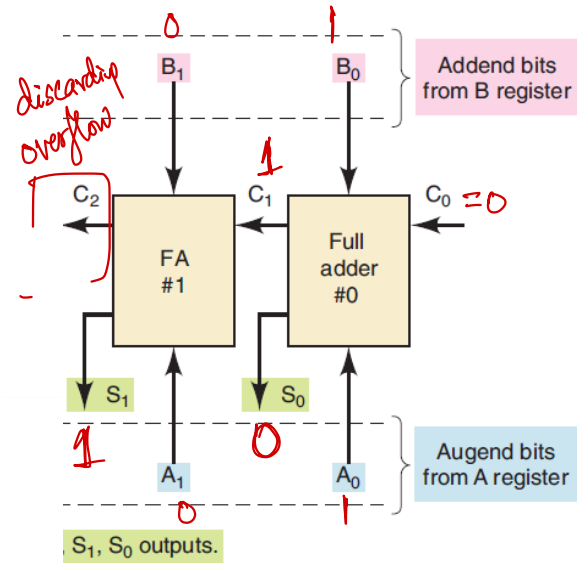
$$Y = \overline{A \oplus B}$$
$$= \overline{A \cdot B + A \cdot \overline{B}}$$

Need for a Full Adder Circuit



Truth Table for a Full Adder:

Input			Output	
A	B	C _{in}	Sum	Carry
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



Handwritten binary addition showing the need for a Full Adder:

$$\begin{array}{r}
 0101 \\
 + 0110 \\
 \hline
 1011
 \end{array}$$

The result is 1011, where the carry-out (1) is the sum of the carry-in (0) and the carry from the previous stage (1).

FA#0

FA#1

Concepts Review (2)

Input			Output	
A	B	Cin	Sum	Carry
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

How to find logic expression for Sum and Carry in terms of A, B and Cin (SOP?)

$$\bar{A} \bar{B} \bar{C}_{in}$$
$$\bar{A}\bar{B}C$$
 $\overline{A} B \overline{C}$
$$S = \bar{A}\bar{B}C + \bar{A}B\bar{C}$$
$$+ A \overline{B} \overline{C} + ABC$$
$$\text{Carry} = \bar{A}BC + A\bar{B}C + AB\bar{C} + ABC$$

min term

↑ product of input variables

$Y = AB$
 both valves are closed
 water valve
 water valve
 $A = 1$
 $B = 1$
 $Y = AB$
 $A \rightarrow$
 $B \rightarrow$
 $Y = 1$ (water is coming)
 $\bar{Y} = \bar{A} + \bar{B}$
 $\bar{Y} \neq \bar{AB}$
 $\bar{A}$
 $\bar{B}$
 $\bar{Y}$
 $Y = 0$ (when is water not coming)

Concepts Review (2)

$$\overline{A + \overline{B \cdot C}}$$

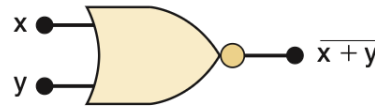
minterms

In Product of Sums:

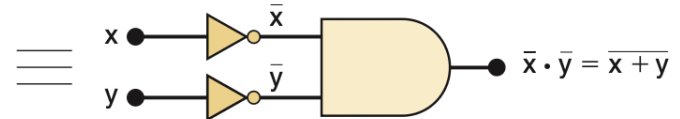
$$\begin{aligned} &= \overline{A} \cdot (\overline{B \cdot C}) \\ &= \overline{A} \cdot (\overline{B} + \overline{C}) \end{aligned}$$

$M_0 = \overline{m_0}$

$$\begin{aligned} &\overline{A + B} \\ &= \overline{A} \cdot \overline{B} \end{aligned}$$

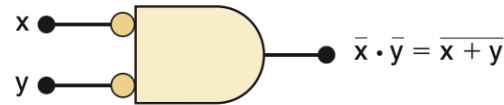


NOR

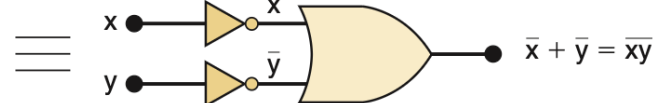


(a)

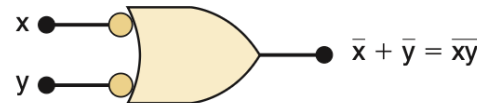
NOT AND



Generic Operations: AND, OR and NOT.



(a)



$$\overline{(A + BC) \cdot (D + EF)}$$

In Sum of Products:

$$\begin{aligned} &\overline{(A + BC) \cdot (D + EF)} \\ &= \overline{(A + BC)} + \overline{(D + EF)} \\ &= (\overline{A} \cdot \overline{BC}) + (\overline{D} \cdot \overline{EF}) \end{aligned}$$

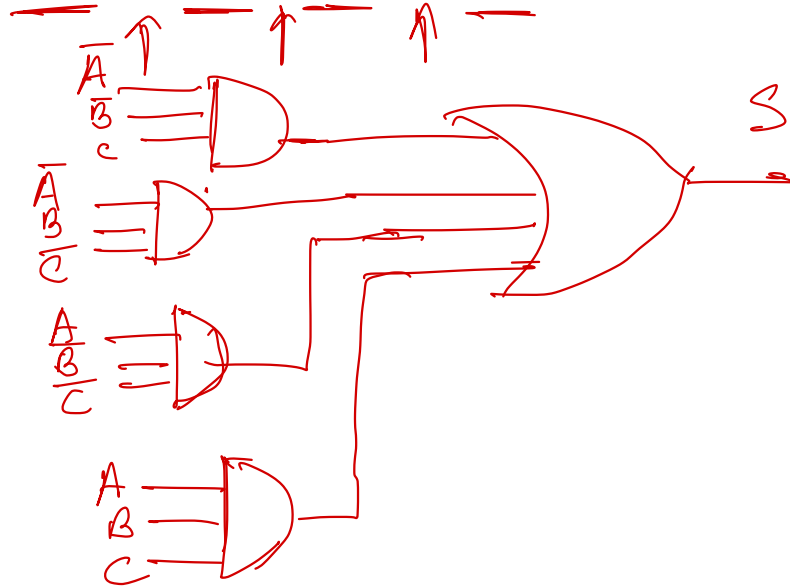
$$\begin{aligned} &\overline{A \cdot B} \\ &= \overline{A} + \overline{B} \end{aligned}$$

Concepts Review (3)

What will it take to implement these expressions (Free to do it the way you want)?

$$Sum = \bar{A}\bar{B}C + \bar{A}B\bar{C} + A\bar{B}\bar{C} + ABC$$

$$Cout = \bar{A}BC + A\bar{B}C + ABC\bar{C} + ABC$$



Concepts Review (4)

Can they be minimised further?

$$Sum = \bar{A}\bar{B}C + \bar{A}B\bar{C} + A\bar{B}\bar{C} + ABC$$

$$= C(\bar{A}\bar{B} + A\bar{B}) + \bar{C}(\bar{A}B + AB)$$

$$= C(\overline{A \oplus B}) + \bar{C}(A \oplus B)$$

$$= C\bar{D} + \bar{C}D$$

$$= C \oplus D = A \oplus B \oplus C$$

$$\begin{array}{ccc} 0 & 0 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{array}$$

$$D = A \oplus B$$

$$Cout = \bar{A}BC + A\bar{B}C + ABC + AB\bar{C}$$

$$= BC + A\bar{B}C + AB\bar{C}$$

$$= C(\bar{B} + BA) + AB\bar{C}$$

$$= C(B + A) + AB\bar{C}$$

$$= BC + AC + AB\bar{C}$$

$$= BC + A(C + \bar{C}B)$$

$$= BC + AC + AB$$

$$= C(A + B) + AB$$

$$= C(A \oplus B + AB) + AB$$

$$= C(A \oplus B) + AB(1 + C)$$

$$= C(A \oplus B) + AB$$

$$= CD + AB$$

Concepts Review (4)

Logic Expression Minimization Using Boolean Algebra

$$A + BC = (A + B) \cdot (A + C)$$

$$Z = AB + A\bar{B} \Rightarrow A(B + \bar{B}) = A$$

$$Z = \underline{B} + A\underline{\bar{B}} \Rightarrow (B + \bar{B}) \cdot (B + A) = A + B$$

$$\text{or } Z = A + \bar{A}B \Rightarrow (A + \bar{A}) \cdot (A + B) = A + B$$

$$Z = A(A + B) \Rightarrow \underline{A}A + AB = A + AB = A(1 + B) = A$$

$$Z = \underline{ABC} + AB\bar{C} + \bar{A}B \Rightarrow AB(C + \bar{C}) + \bar{A}B = AB + \bar{A}B = B$$

Concepts Review (4)

Logic Expression Minimization Using K-Map

$$Sum = \bar{A}\bar{B}C + \bar{A}B\bar{C} + A\bar{B}\bar{C} + ABC$$

$$Cout = \bar{A}BC + A\bar{B}C + AB\bar{C} + ABC$$

Handwritten labels above the table: $\bar{A}\bar{B}$, $\bar{A}B$, $A\bar{B}$, AB

AB →	00	01	11	10
$\bar{C}$ 0	0	1	0	1
C 1	1	0	1	0

Here $C = C_{in}$

$$\begin{aligned} \text{SUM} &= \bar{A}\bar{B}C + \bar{A}B\bar{C} + A\bar{B}\bar{C} + ABC \\ &= A \oplus B \oplus C \end{aligned}$$

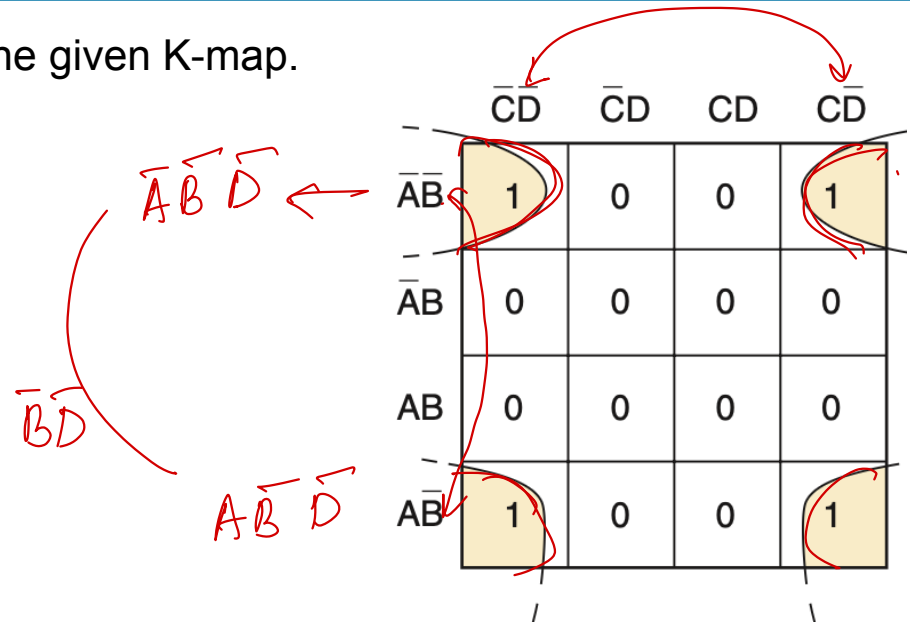
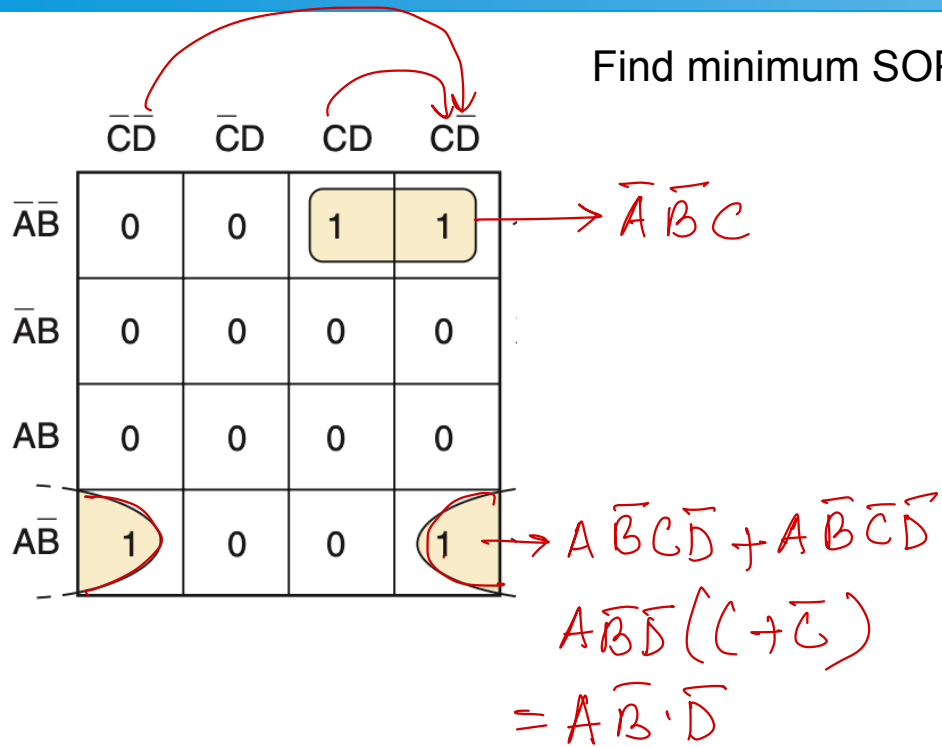
Handwritten label above the table: AB

AB →	00	01	11	10
0	0	0	1	0
1	0	1	1	1

$$\begin{aligned} \text{CARRY} &= C_{in}B + AB + C_{in}A \\ &= (A \oplus B)C + AB \end{aligned}$$

Concepts Review (4)

Find minimum SOP from the given K-map.



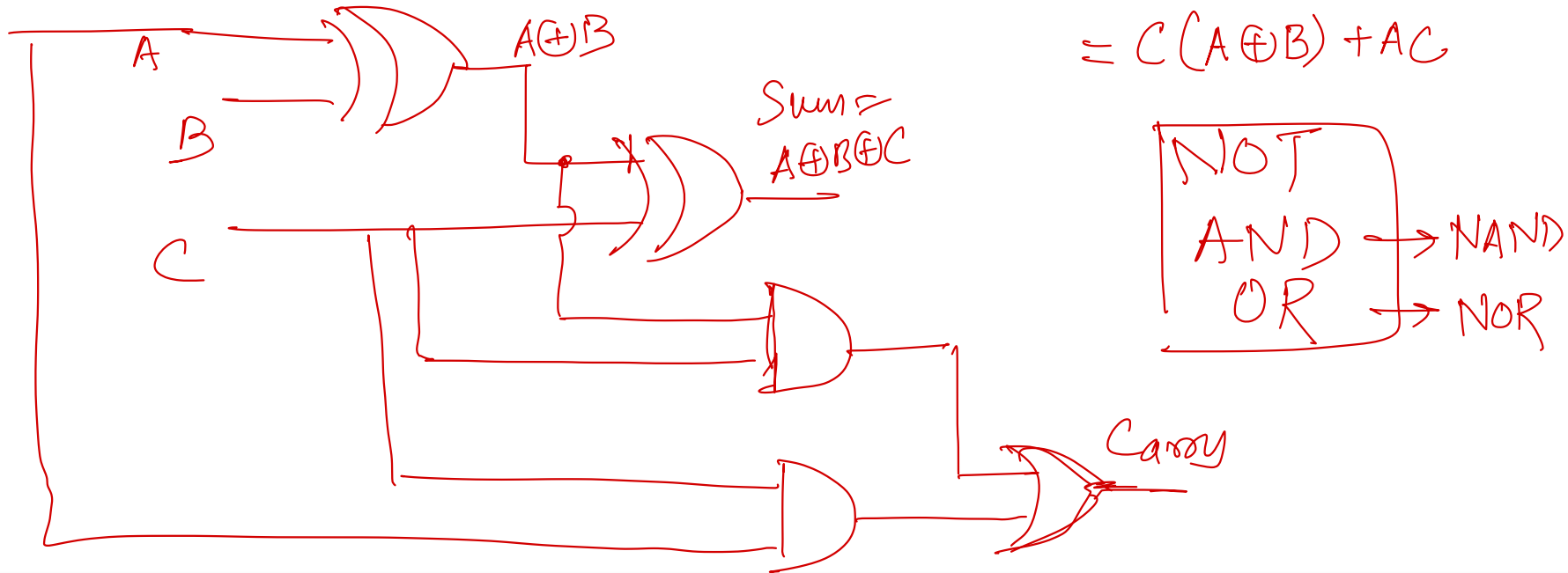
Concepts Review (5)

Implementation of these minimised expressions through any Logic Gates

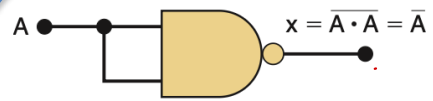
$$Sum = A \oplus B \oplus C$$

$$Carry = AB + BC_{in} + AC_{in}$$

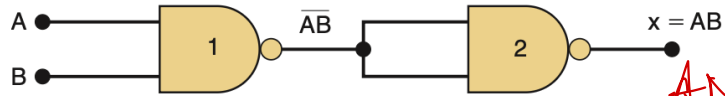
$$= C(A \oplus B) + AC$$



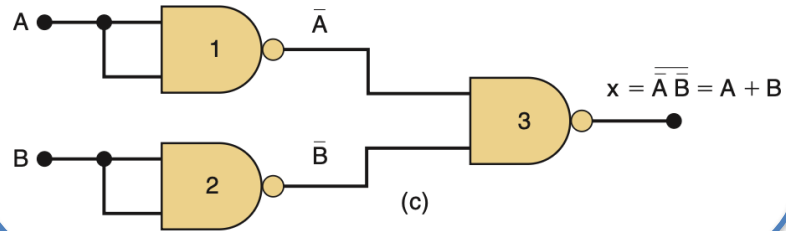
Concepts Review (5)



(a)

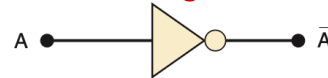


(b)

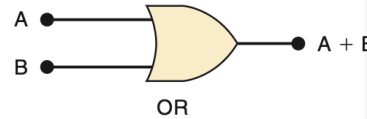


(c)

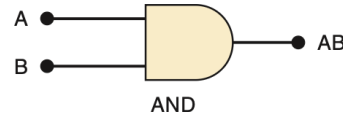
Universal gates



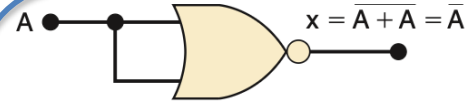
INVERTER



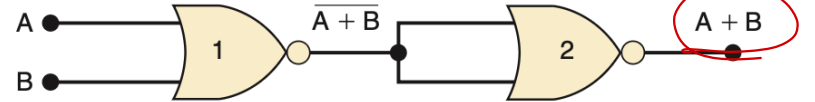
OR



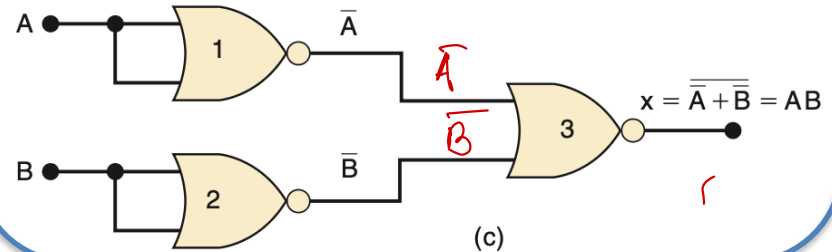
AND



(a)



(b)



(c)

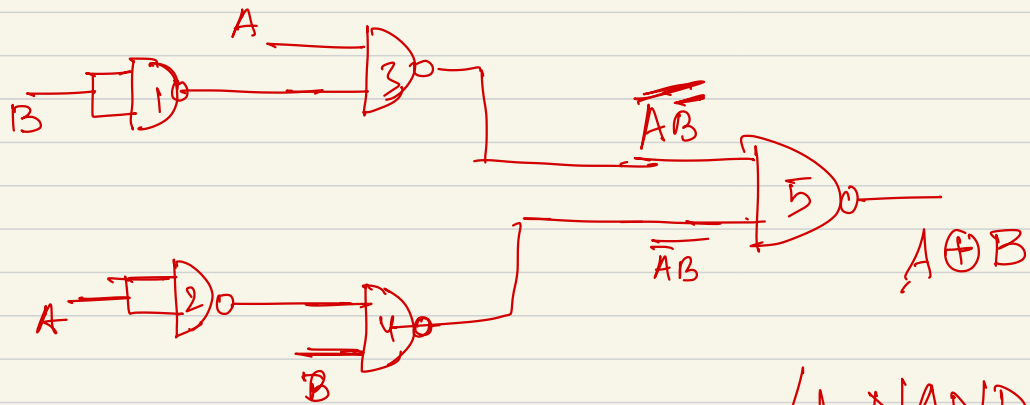


Implement XOR using either NAND gates or NOR gates

$$\text{XOR} - \overline{A\bar{B} + \bar{A}B} = \overline{A\bar{B}} \cdot \overline{\bar{A}B}$$

NAND

$\overline{A\bar{B}} \rightarrow \text{DB}$
 $\overline{\bar{A}B} \rightarrow \text{DA}$

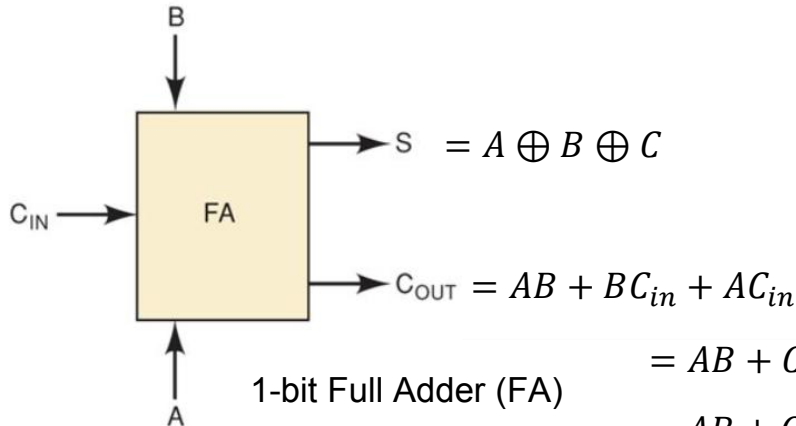
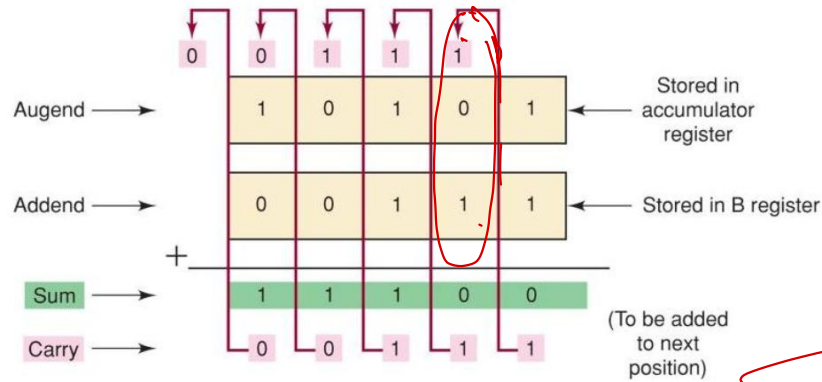


4 NAND gate



Summary (Computation in Processors)

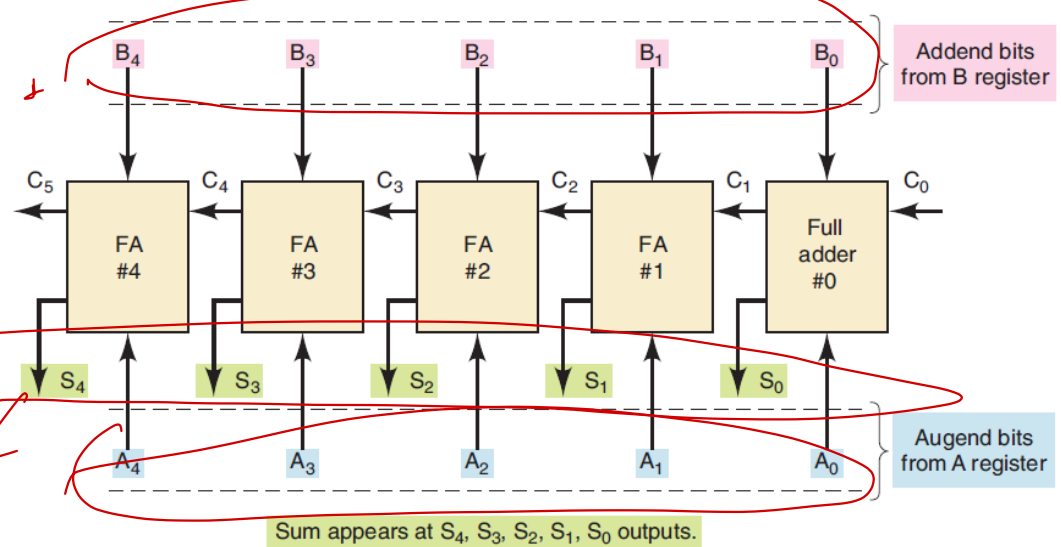
Arithmetic Operation: Addition



1-bit Full Adder (FA)

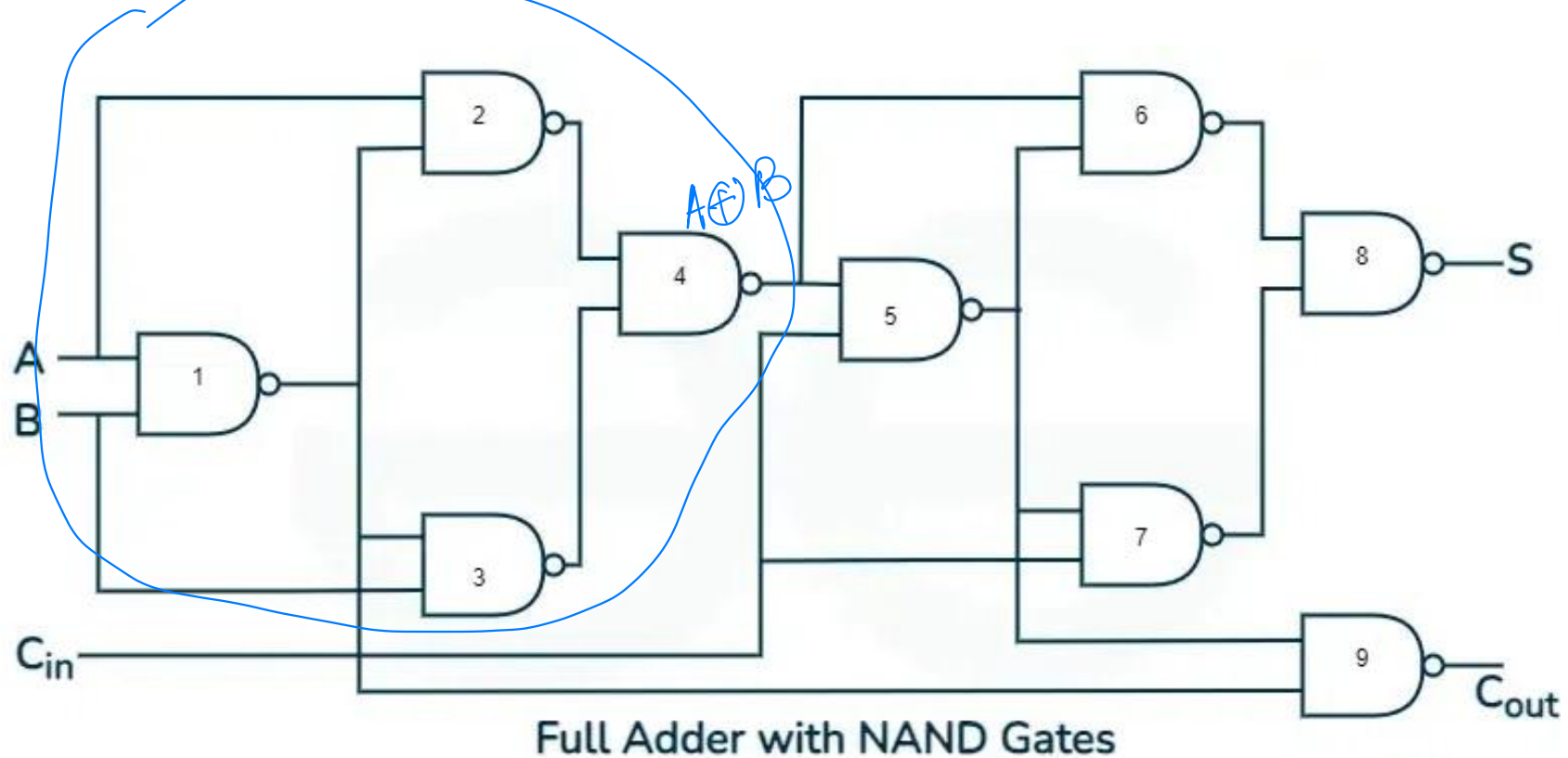
$$\begin{aligned}
 &= AB + C_{in}(A + B) = AB + C_{in}((A \oplus B) + AB) = AB + ABC_{in} + C_{in}(A \oplus B) \\
 &= AB + C_{in}(A \oplus B) \quad \text{Can use same gate as present in Sum (S).}
 \end{aligned}$$

5-bit Addition using five Full Adder (FA)



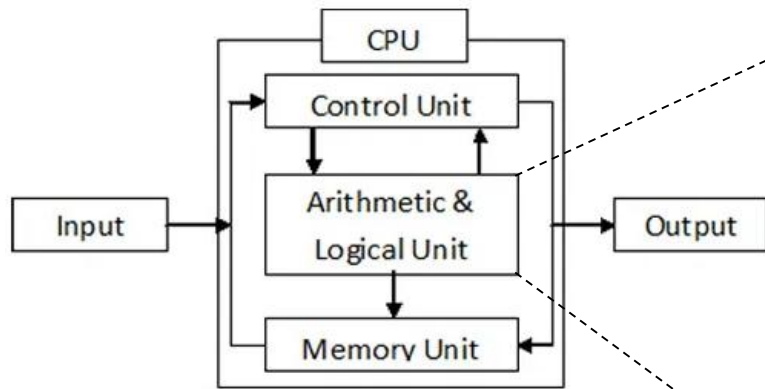
5-bit

Implementation of 1-bit Full-Adder circuit

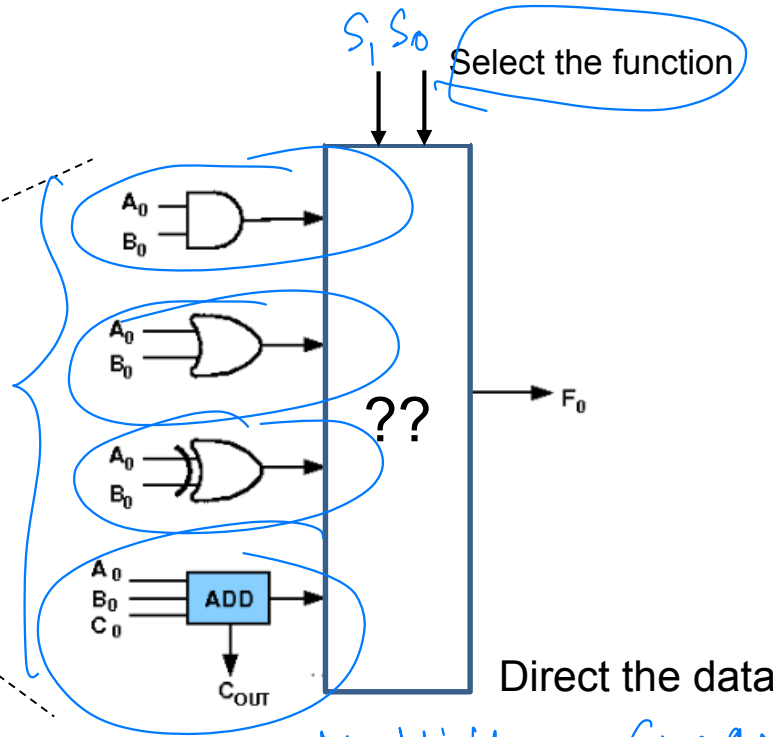


Compute unit in *computers*: Arithmetic Logic Unit (ALU)

Block diagram of a *digital computer*



Example: 1-bit ALU unit



S₁ S₀
0 0
0 1
1 0
1 1

Truth Table

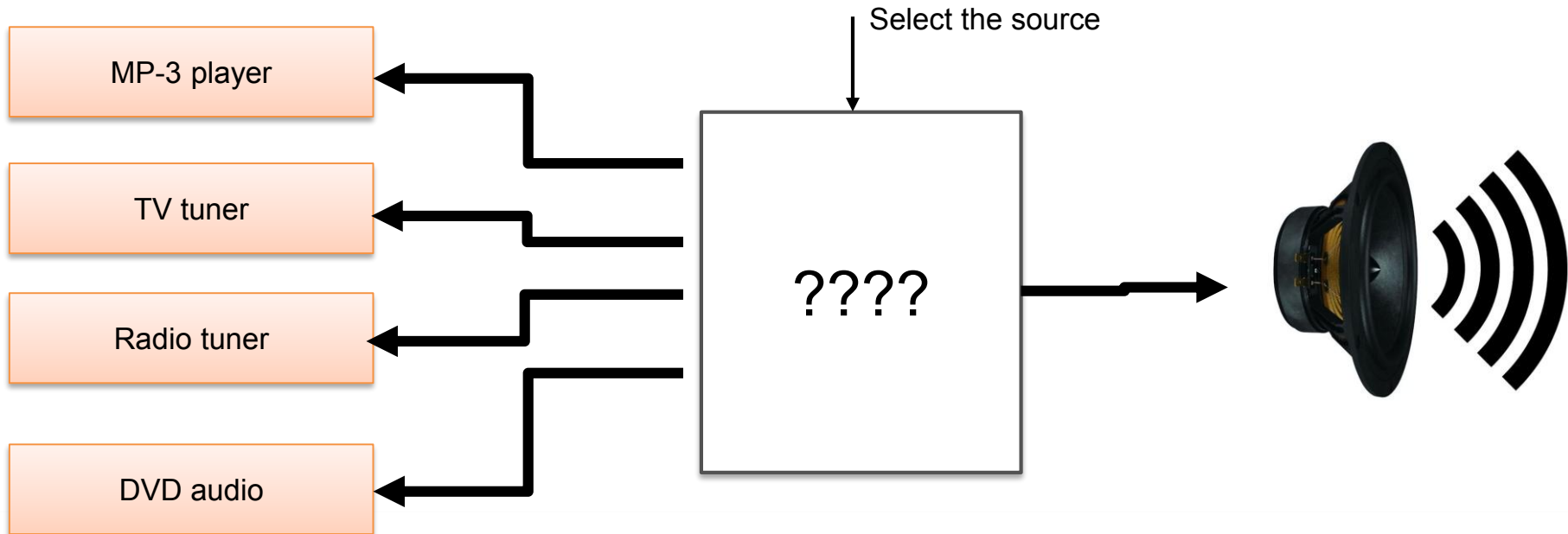
Direct the data
Multiplexer (4:1)
Mux

Select
 $\log_2 N$

Practical example

This unknown *unit* selects music from one of the different sources present and sends it to the speakers.

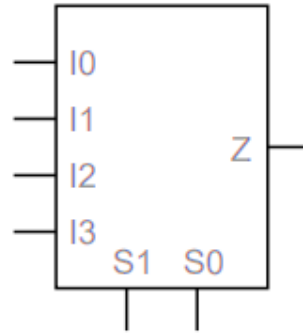
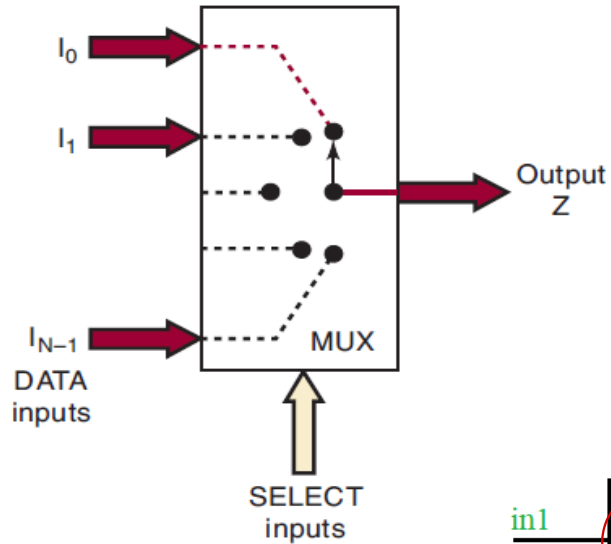
Modern home stereo system



Multiplexer

- A **multiplexer (MUX)** selects 1 of N input data sources and transmits the selected data to a single output - called multiplexing.

Example: (N = 4 data sources and $\log_2 N$ select lines)

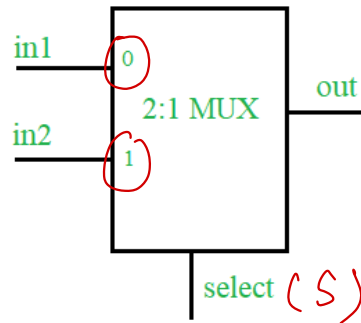


S_1	S_0	Z
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

Expression:

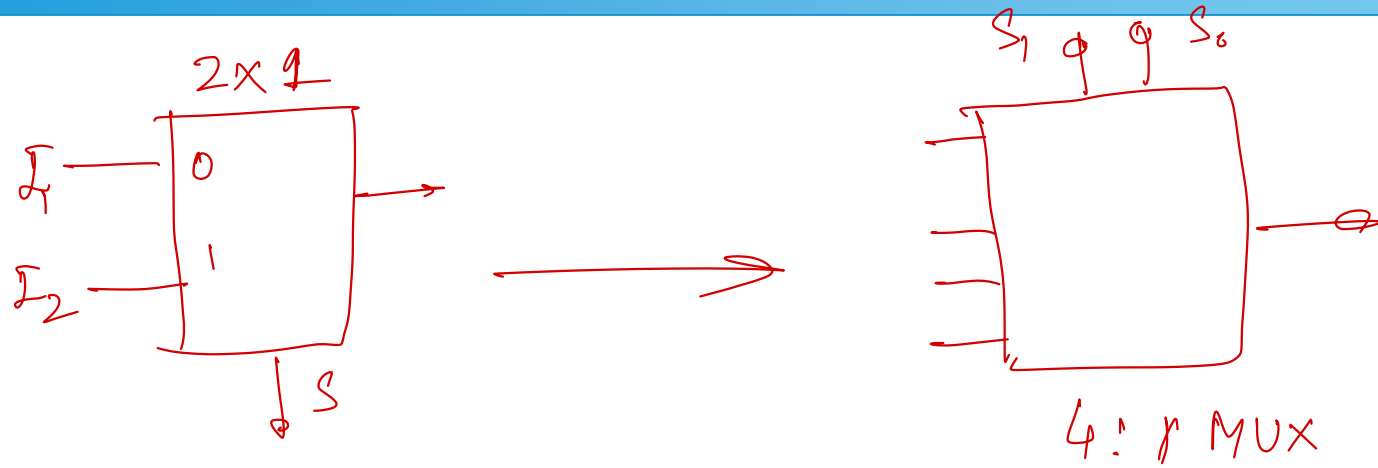
$$Z = I_0 \bar{S}_1 \bar{S}_0 + I_1 \bar{S}_1 S_0 + I_2 S_1 \bar{S}_0 + I_3 S_1 S_0$$

Write the Truth table and Logic Expression (out)



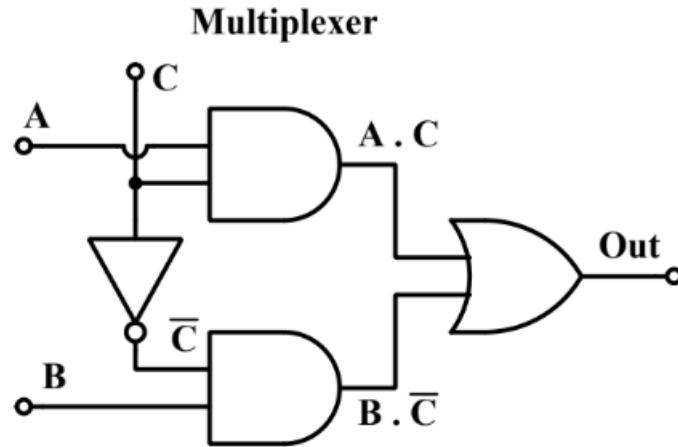
$$Z = in1 \times \bar{S} + in2 \times S$$

A 4-to-1 Multiplexer using 2-to-1 Multiplexer



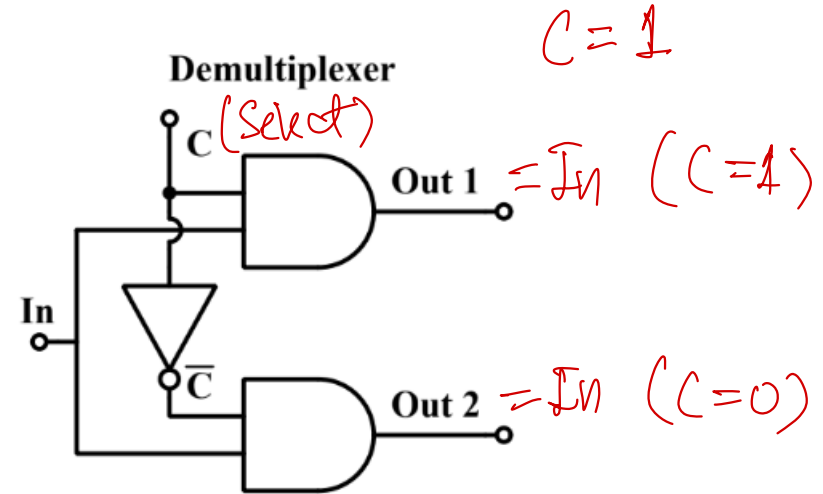
DeMultiplexers (Opposite of Multiplexers)

- Demultiplexers do the opposite. The input data comes on a single line and depending on a line number code, it is put on one out of multiple output lines.



2 : 1

C

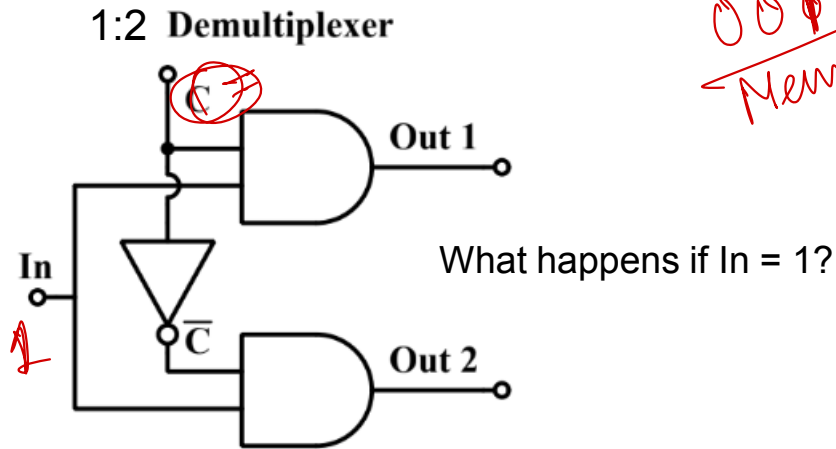


1 < 2

1:2 DeMux

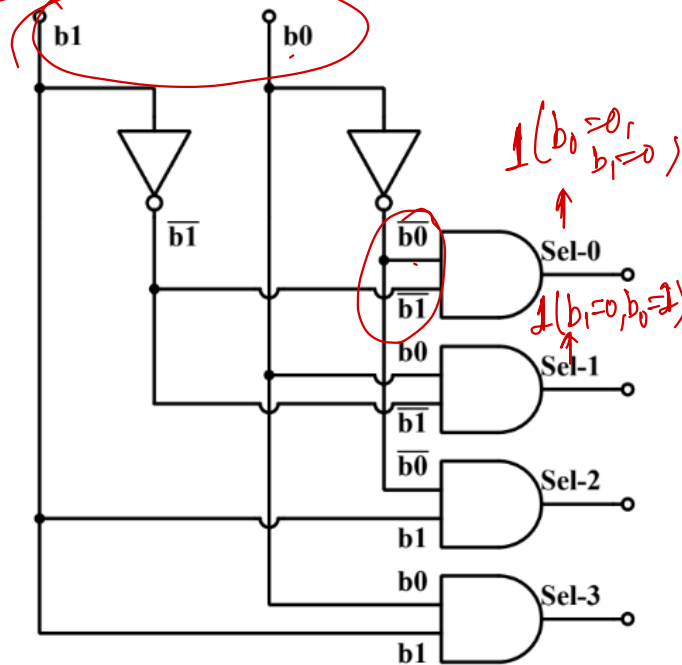
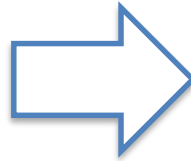
Another level of coding (decoding)

- What can we do if we want to multiplex from or demultiplex to more than two lines?
- We use multi-bit decoders which provide select lines to replace C and $\bar{C}$ in circuit on the left.



001
Memory

Decoders



- The circuit on the right shows a 2 bit decoder which provides 4 select lines. The select signals replace C and $\bar{C}$ when we have more than 2 lines to multiplex or demultiplex.
- Apart from their use in multiplexers and demultiplexers, these are widely used for **Memory address decoding** and enabling selected devices connected to processors etc.